

BÀI THỰC HÀNH DEEP LEARNING CHO KHOA HỌC DỮ LIỆU

LAB 1. Các Khái Niệm Cơ Bản

Hà Minh Tuấn
Khoa Toán - Tin học, ĐHKHTN, ĐHQG-HCM

Ngày 17 tháng 3 năm 2026

Bài Tập Thực Hành: PyTorch Cơ Bản

Các bài tập được sắp xếp theo thứ tự từ dễ đến khó, bám sát nội dung notebook `pytorch_basics.ipynb`. Mỗi bài yêu cầu viết code, dự đoán kết quả, hoặc giải thích khái niệm – đọc kỹ trước khi code.

Phần 1 – Tensor: Tạo và Thuộc Tính

Bài 1. Tạo tensor và kiểm tra thuộc tính. Cho đoạn code sau:

```
import torch

a = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])
b = torch.zeros(3, 3)
c = torch.randn(2, 4)
```

Nhiệm vụ:

1. Điền vào bảng dưới đây *trước khi chạy code*:

Tensor	.shape	.dim()	.dtype	.numel()
a				
b				
c				

2. Chạy code và đối chiếu. Nếu có ô sai, giải thích tại sao.
3. `.shape` và `.size()` có khác nhau không? Thử `a.size(0)` và `a.shape[0]` – kết quả là gì?

Ô trả lời:

Bài 2. Tạo tensor theo yêu cầu – không dùng giá trị cứng. Viết code tạo ra các tensor sau, mỗi yêu cầu chỉ dùng *một* lời gọi hàm *PyTorch*:

1. Ma trận đơn vị 5×5 .
2. Vector gồm 8 số cách đều nhau từ 0 đến 1 (đầu và cuối đều có).
3. Tensor shape $(3, 4)$ với mọi phần tử bằng -1 .
4. Tensor shape $(2, 3, 4)$ với các số nguyên từ 0 đến 23 (theo thứ tự tăng dần, dùng `arange` rồi `reshape`).
5. Tensor shape $(4, 4)$ mà đường chéo chính là $[1.0, 2.0, 3.0, 4.0]$, còn lại bằng 0. *Gợi ý: xem `torch.diag`.*

```
import torch

# (a)

# (b)

# (c)

# (d)

# (e)
```

Bài 3. Indexing và Slicing – dự đoán trước khi chạy. Cho tensor:

```
import torch
x = torch.arange(1, 17).reshape(4, 4)
print(x)
```

Trước khi chạy, viết ra kết quả bạn kỳ vọng của từng biểu thức:

Biểu thức	Dự đoán	Kết quả thực tế
<code>x[1, 2]</code>		
<code>x[:, 3]</code>		
<code>x[2:4, 1:3]</code>		
<code>x[x % 3 == 0]</code>		
<code>x[-1, -1]</code>		
<code>x[[0, 2], :]</code>		

Sau khi chạy: trường hợp nào bạn dự đoán sai? Giải thích nguyên nhân.

Phần 2 – Phép Toán Trên Tensor

Bài 4. In-place vs out-of-place – phân biệt bộ nhớ. *Đọc code sau và trả lời trước khi chạy:*

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([10.0, 20.0, 30.0])

# Phiên bản out-of-place
c = a.add(b)
print(f"a sau add (out-of-place): {a}")
print(f"c = {c}")

# Phiên bản in-place
a.add_(b)
print(f"a sau add_ (in-place): {a}")
```

1. Giá trị của `a` sau dòng `c = a.add(b)` là bao nhiêu? Giải thích tại sao.
2. Giá trị của `a` sau dòng `a.add_(b)` là bao nhiêu?
3. Khi nào nên dùng in-place? Khi nào *không* được dùng in-place? *Gợi ý: thử gán `requires_grad=True` cho `a` rồi gọi `a.add_(b)` – điều gì xảy ra?*

Bài 5. Phép toán ma trận – phân biệt $*$, $@$, `matmul`. Cho hai ma trận:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

1. Tính tay (không dùng máy tính) kết quả của: $A * B$ (element-wise), $A \cdot B$ (nhân ma trận), A^T .
2. Viết code PyTorch để kiểm chứng bằng cả hai cách: `torch.matmul(A, B)` và `A @ B`. Kết quả có giống nhau không?
3. Thêm chiều batch: tạo `A_batch` shape (3, 2, 2) bằng cách xếp chồng `A` ba lần (`torch.stack`), rồi nhân với `B`. Shape của kết quả là gì? Giải thích.

```
import torch

A = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
B = torch.tensor([[5.0, 6.0], [7.0, 8.0]])

# (a) Tính tay:
# A * B =
# A @ B =
# A.t() =

# (b) Kiểm chứng bằng code:

# (c) Batch matmul:
```

Bài 6. Reduction operations – hiểu chiều `dim`. Đây là nguồn gây nhầm lẫn phổ biến. Cho tensor:

```
import torch
x = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0],
                  [7.0, 8.0, 9.0]])
```

1. Điền vào bảng *trước khi chạy*. Với mỗi biểu thức, ghi rõ cả **giá trị** và **shape** của kết quả:

Biểu thức	Dự đoán (giá trị + shape)	Kết quả thực tế
<code>x.sum()</code>		
<code>x.sum(dim=0)</code>		
<code>x.sum(dim=1)</code>		
<code>x.sum(dim=1, keepdim=True)</code>		
<code>x.max(dim=0).values</code>		
<code>x.argmax(dim=1)</code>		

- Giải thích bằng lời: `dim=0` nghĩa là “rút gọn theo chiều nào”? Vẽ sơ đồ mũi tên để minh hoạ sự khác biệt giữa `dim=0` và `dim=1`.

Bài 7. Reshape, view, squeeze, unsqueeze – tìm lỗi. Code dưới đây có **2 lỗi**. Tìm và sửa mà không cần chạy thử.

```
import torch

x = torch.arange(12)          # shape: (12,)

# Muốn reshape thành (3, 4)
y = x.reshape(3, 5)          # Dòng A

# Muốn thêm chiều đầu: (1, 3, 4)
z = y.unsqueeze(0)

# Muốn bỏ chiều đầu: (3, 4)
w = z.squeeze(1)              # Dòng B

print(y.shape, z.shape, w.shape)
```

- Dòng nào sai? Giải thích lý do toán học.
- Viết lại đoạn code đúng.
- `view` và `reshape` khác nhau điểm gì? Khi nào `view` báo lỗi nhưng `reshape` không? Gọi ý: thử `x.t().view(-1)` – điều gì xảy ra?

Phần 3 – NumPy Integration và Bộ Nhớ Dùng Chung

Bài 8. Shared memory – hành vi nguy hiểm tiềm ẩn. Đây là khái niệm quan trọng, dễ gây bug khó phát hiện.

```
import torch
import numpy as np

# Trường hợp 1: from_numpy (chia sẻ bộ nhớ)
np_a = np.array([1.0, 2.0, 3.0])
t_a = torch.from_numpy(np_a)

np_a[0] = 999.0
print(f"t_a sau khi sửa np_a: {t_a}")    # Dòng X

# Trường hợp 2: torch.tensor (KHÔNG chia sẻ)
np_b = np.array([4.0, 5.0, 6.0])
t_b = torch.tensor(np_b)

np_b[0] = 999.0
print(f"t_b sau khi sửa np_b: {t_b}")    # Dòng Y
```

1. Trước khi chạy: dự đoán giá trị in ra ở Dòng X và Dòng Y.
2. Giải thích cơ chế: tại sao `from_numpy` chia sẻ bộ nhớ còn `torch.tensor` thì không?
3. Trong thực tế huấn luyện mạng nơ-ron, việc chia sẻ bộ nhớ này có thể gây ra bug như thế nào? Nêu một kịch bản cụ thể.
4. Nếu muốn chuyển tensor sang NumPy mà không chia sẻ bộ nhớ, bạn dùng cách nào?

Bài 9. Chuyển đổi tensor – các trường hợp đặc biệt. Thực hiện và giải thích từng trường hợp:

```
import torch
import numpy as np

# Trường hợp 1: tensor với requires_grad
```

```

t1 = torch.tensor([1.0, 2.0], requires_grad=True)
# t1.numpy()      # <-- dòng này báo lỗi gì? Cách fix?

# Trường hợp 2: tensor trên GPU (nếu có)
# t2 = torch.tensor([1.0, 2.0]).cuda()
# t2.numpy()      # <-- dòng này báo lỗi gì? Cách fix?

# Trường hợp 3: tensor kiểu int
t3 = torch.tensor([1, 2, 3], dtype=torch.int32)
arr = t3.numpy()
print(f"dtype của NumPy array: {arr.dtype}")

```

1. Với `t1`: tại sao không thể gọi `.numpy()` trực tiếp? Viết đoạn fix đúng (2 cách khác nhau).
2. Với `t2`: tại sao tensor GPU không chuyển thẳng sang NumPy được? Viết đoạn fix.
3. `t3.numpy()` thành công – dtype của array là gì? Thay `int32` thành `int64` thì dtype NumPy thay đổi không?

Phần 4 – Autograd và Đồ Thị Tính Toán

Bài 10. Tính gradient thủ công – kiểm chứng với PyTorch. Cho hàm số $f(x, y) = 3x^2 + 2xy - y^3$ tại điểm $(x, y) = (1, 2)$.

1. Tính tay $\frac{\partial f}{\partial x}$ và $\frac{\partial f}{\partial y}$ tại $(1, 2)$. Viết rõ từng bước.

2. Viết code PyTorch để kiểm chứng:

```

import torch

x = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(2.0, requires_grad=True)

# Tính f
f = 3 * x**2 + 2 * x * y - y**3

# Tính gradient
f.backward()

print(f"df/dx = {x.grad}")
print(f"df/dy = {y.grad}")

```

3. Kết quả có khớp không? Nếu không, tìm lỗi trong phần tính tay.
4. Bây giờ muốn tính $\frac{\partial^2 f}{\partial x^2}$ (đạo hàm bậc hai). PyTorch có hỗ trợ không? Thử `torch.autograd.grad` với `create_graph=True`.

Bài 11. Gradient tích lũy – lỗi phổ biến nhất. Code dưới đây có lỗi logic dẫn đến gradient sai. Tìm lỗi, giải thích cơ chế, và sửa.

```
import torch

x = torch.tensor(2.0, requires_grad=True)

# Muốn tính tổng gradient qua 3 lần forward khác nhau
tong_grad = 0.0
for i in range(1, 4):
    f = i * x**2          # f1=x^2, f2=2x^2, f3=3x^2
    f.backward()
    tong_grad += x.grad.item()
    print(f"Lần {i}: x.grad = {x.grad.item():.1f}")

print(f"Tổng gradient (mong muốn: 4+8+12=24): {tong_grad:.1f}")
```

1. Chạy code và quan sát kết quả thực tế. Tổng gradient in ra là bao nhiêu? Có đúng không?
2. Giải thích cơ chế tích lũy gradient của PyTorch. Tại sao gradient “bị cộng dồn” thay vì được ghi đè?
3. Sửa code để in ra gradient đúng ở mỗi lần và tính tổng chính xác.

```
import torch

x = torch.tensor(2.0, requires_grad=True)
tong_grad = 0.0

for i in range(1, 4):
    f = i * x**2
    f.backward()
    tong_grad += x.grad.item()
    # Thêm dòng fix vào đây:

    print(f"Lần {i}: x.grad = {x.grad.item():.1f}")

print(f"Tổng gradient: {tong_grad:.1f}")
```

Bài 12. detach() và torch.no_grad() – khi nào dùng cái nào.


```

import torch

x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
w = torch.tensor([0.5, 0.5, 0.5], requires_grad=True)

# Tính loss
y = (x * w).sum()
y.backward()

# Muốn dùng kết quả gradient để tính thêm
# mà KHÔNG muốn đưa vào đồ thị tính toán
grad_x = x.grad          # Cách 1
grad_x_d = x.grad.detach() # Cách 2

# Kiểm tra
print(f"requires_grad của x.grad: {x.grad.requires_grad}")
print(f"requires_grad sau detach: {grad_x_d.requires_grad}")

# Với torch.no_grad
with torch.no_grad():
    ket_qua = x * 2.0    # Phép tính này có requires_grad không?
    print(f"requires_grad trong no_grad: {ket_qua.requires_grad}")

```

1. Khi nào nên dùng `.detach()`? Khi nào nên dùng `torch.no_grad()`? Chúng có thể dùng thay thế nhau không?
2. Trong vòng lặp đánh giá (evaluation/inference), tại sao ta thường bao toàn bộ code bằng `with torch.no_grad()`? Lợi ích về bộ nhớ là gì?
3. `x.grad.requires_grad` in ra `False`. Có nghĩa là gì? Gradient của gradient (higher-order derivative) có tính được không?

Phần 5 – Tổng Hợp và Ứng Dụng

Bài 13. Hiện thực Gradient Descent từ đầu – chỉ dùng tensor. Dùng PyTorch thuần (không dùng `nn.Module`, không dùng `optim`) để tìm nghiệm của bài toán tối ưu:

$$\min_{w \in \mathbb{R}} L(w) = (w - 3)^2 + 2(w - 3) + 1$$

1. Tính tay nghiệm tối ưu w^* (đạo hàm bằng 0).
2. Hoàn thành code dưới đây – chỉ điền vào các dòng có `# ???`:

```

import torch
import matplotlib.pyplot as plt

w = torch.tensor(0.0, requires_grad=True)
lr = 0.1
lich_su_loss = []
lich_su_w = []

for buoc in range(50):
    # ??? Tính L(w)
    L = None

    # ??? Xoá gradient cũ
    # ...

    # ??? Tính gradient
    # ...

    # ??? Cập nhật w (in-place, không phá vỡ đồ thị)
    with torch.no_grad():
        w -= None # ???

    lich_su_loss.append(L.item())
    lich_su_w.append(w.item())

print(f"w tối ưu tìm được: {w.item():.4f} (kỳ vọng: {2.0})")

# Vẽ đường hội tụ
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(lich_su_loss); axes[0].set_title("Loss theo bước")
axes[0].set_xlabel("Bước"); axes[0].set_ylabel("L(w)")
axes[1].plot(lich_su_w); axes[1].set_title("w theo bước")
axes[1].axhline(2.0, color="red", ls="--", label="w*=2")
axes[1].legend()
plt.tight_layout(); plt.show()

```

- Thay $lr = 0.1$ bằng $lr = 1.5$ rồi chạy lại. Điều gì xảy ra? Giải thích bằng khái niệm learning rate và hội tụ.

Bài 14. Hàm Sigmoid và đạo hàm – kết nối autograd với toán học.

- Viết code tính $\sigma(z) = \frac{1}{1 + e^{-z}}$ không dùng `torch.sigmoid`, rồi dùng autograd tính $\sigma'(z)$ tại $z = 0, 1, 2, 5$.

```

import torch

diem_kiem_tra = [0.0, 1.0, 2.0, 5.0]
print(f"{'z':>6} {'sigma(z)':>10} {'sigma_prime(z) [autograd]':>26}"
      f" {'sigma_prime(z) [tay]':>22}")

for z_val in diem_kiem_tra:
    z = torch.tensor(z_val, requires_grad=True)

```

```

# ??? Tính  $\sigma(z)$  không dùng torch.sigmoid
sigma = None

# ??? Tính đạo hàm bằng autograd
sigma.backward()
grad_autograd = z.grad.item()

# ??? Tính đạo hàm bằng công thức tay:  $\sigma(z)*(1-\sigma(z))$ 
grad_tay = None

print(f"{z_val:>6.1f} {sigma.item():>10.6f}"
      f" {grad_autograd:>26.6f} {grad_tay:>22.6f}")

```

2. Với $z = 5$: giá trị $\sigma'(5)$ có nhỏ không? Điều này liên quan đến vấn đề gì trong huấn luyện mạng sâu?
3. So sánh `grad_autograd` và `grad_tay` – chúng có khớp nhau không? Sai số tối đa là bao nhiêu?

Bài 15. Mini-project: Hồi Quy Tuyến Tính Bằng Gradient Descent Thuần. Dùng kiến thức tổng hợp từ tất cả các bài trước để tự xây dựng bộ giải hồi quy tuyến tính $\hat{y} = wx + b$ bằng gradient descent, *chỉ dùng `tensor` và `autograd`, không dùng bất kỳ module nào khác.*

```

import torch
import numpy as np
import matplotlib.pyplot as plt

# ---- Dữ liệu tổng hợp (đừng thay đổi) ----
torch.manual_seed(42)
X = torch.linspace(-3, 3, 100)          # shape: (100,)
y_true = 2.5 * X - 1.0                  # w_that=2.5, b_that=-1.0
y = y_true + torch.randn(100) * 0.5    # thêm nhiễu

# ---- Khởi tạo tham số ----
w = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

# ---- Vòng lặp huấn luyện ----
lr = 0.01
so_epoch = 200
lich_su = []

for epoch in range(so_epoch):
    # ??? Bước 1: Tính  $\hat{y} = w * X + b$ 
    y_hat = None

    # ??? Bước 2: Tính MSE loss =  $\text{mean}((\hat{y} - y)^2)$ 
    loss = None

    # ??? Bước 3: Tính gradient
    # ...

    # ??? Bước 4: Cập nhật  $w, b$  (dùng torch.no_grad)
    with torch.no_grad():

```

```

        # ...
        pass

# ??? Bước 5: Xoá gradient
# ...

if epoch % 20 == 0:
    lich_su.append((epoch, loss.item(), w.item(), b.item()))
    print(f"Epoch {epoch:4d}: loss={loss.item():.4f}, "
          f"w={w.item():.4f}, b={b.item():.4f}")

print(f"\nKết quả: w={w.item():.4f} (thật: 2.5), "
      f"b={b.item():.4f} (thật: -1.0)")

# ---- Vẽ kết quả ----
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].scatter(X.numpy(), y.numpy(), s=10, alpha=0.5,
                label="Dữ liệu")
axes[0].plot(X.numpy(), (w * X + b).detach().numpy(),
             "r-", lw=2, label=f"Dự đoán (w={w.item():.2f})")
axes[0].plot(X.numpy(), y_true.numpy(),
             "g--", lw=1.5, label="Đường thật")
axes[0].legend(); axes[0].set_title("Hồi quy tuyến tính")

epochs_plot = [e for e, *_ in lich_su]
losses_plot = [l for _, l, *_ in lich_su]
axes[1].plot(epochs_plot, losses_plot, "b-o", markersize=4)
axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("MSE Loss")
axes[1].set_title("Loss curve"); axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("linear_regression.png", dpi=150, bbox_inches="tight")
plt.show()

```

Câu hỏi sau khi hoàn thành:

1. Giá trị w và b tìm được có gần với giá trị thật ($w = 2.5$, $b = -1.0$) không? Nếu chưa gần, hãy điều chỉnh lr hoặc `so_epoch`.
2. Tại sao ở Bước 4 phải bọc trong `torch.no_grad()`? Thử bỏ đi – điều gì xảy ra với đồ thị tính toán?
3. Thay MSE loss bằng MAE loss ($|y_{\hat{}} - y|$). Quá trình hội tụ có thay đổi không? Tại sao MAE khó tối ưu hơn MSE?

Bảng Tiêu Chí Tự Đánh Giá

Bài	Tiêu chí đánh giá	Tự chấm (/10)	Điểm
1	Điền đúng bảng thuộc tính + giải thích shape/size	/10	5
2	Tạo đúng 5 tensor chỉ dùng 1 lời gọi hàm mỗi câu	/10	5
3	Dự đoán đúng indexing + giải thích trường hợp sai	/10	5
4	Phân biệt in-place/out-of-place + thử với grad	/10	7
5	Tính tay đúng + batch matmul + giải thích shape	/10	7
6	Dự đoán đúng dim + vẽ sơ đồ minh hoạ	/10	7
7	Tìm đúng 2 lỗi + giải thích view vs reshape	/10	7
8	Dự đoán shared memory + nêu kịch bản bug	/10	8
9	Xử lý đúng 3 trường hợp đặc biệt khi chuyển đổi	/10	8
10	Tính tay đúng gradient + kiểm chứng + đạo hàm bậc 2	/10	8
11	Tìm lỗi tích lũy gradient + sửa code đúng	/10	8
12	Phân biệt detach vs no_grad + giải thích bộ nhớ	/10	8
13	Code GD hoàn chỉnh + thí nghiệm lr lớn	/10	9
14	Tính sigmoid + đạo hàm + liên hệ vanishing gradient	/10	9
15	Mini-project hoàn chỉnh + trả lời 3 câu hỏi	/10	9
	Tổng		100